

LAB 2

Containerization & Kubernetes Orchestration

Task 1 — Containerize a Sample Application Using Docker

Dockerfile • Image Build • Image Layers • Container Execution • Application Verification

Student	Kunal Prajapati
Application	Node.js HTTP sample application
Application Port	3000
Docker Image	lab2-docker-app:v1
Container	lab2-container
Registry Tag	kunalprajapati1/lab2-docker-app:v1

This report documents the completed Docker containerization workflow and provides screenshot evidence for the Task 1 requirements.

Submission Evidence

The evidence shows the source application and Dockerfile, successful image creation, image-layer history, container startup and port mapping, browser verification, and container logs.

1. Sample Application and Project Files

The project contains a Node.js HTTP application. The application listens on port 3000 and returns a simple HTML response containing the lab title and a successful container-running message.

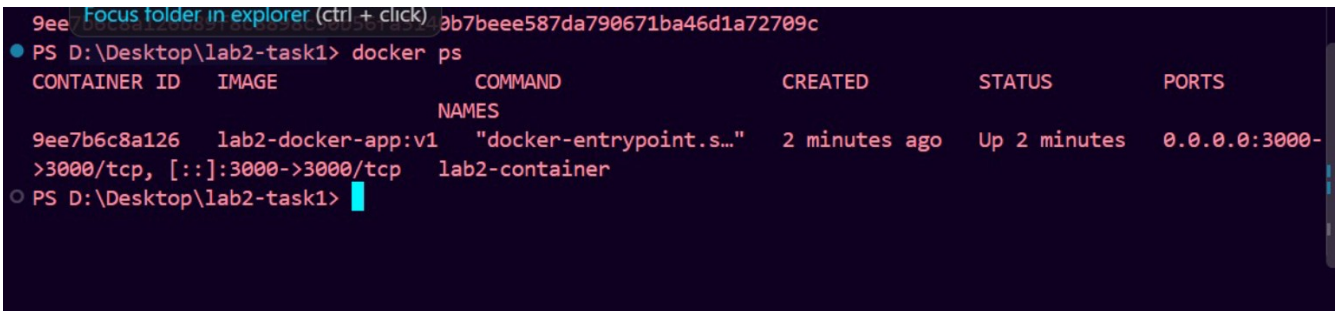


Figure 1. Node.js application (app.js) and project structure.

Key application behavior

- Uses Node.js built-in **http** module to create the server.
- Listens on **0.0.0.0:3000**, allowing access through the published Docker port.
- Returns an HTML page with the message **Hello from Docker!** and lab information.

2. Dockerfile and Image Build

A Dockerfile was used to create the application image from the Node 20 Alpine base image. The build completed successfully and produced the image tag **lab2-docker-app:v1**.

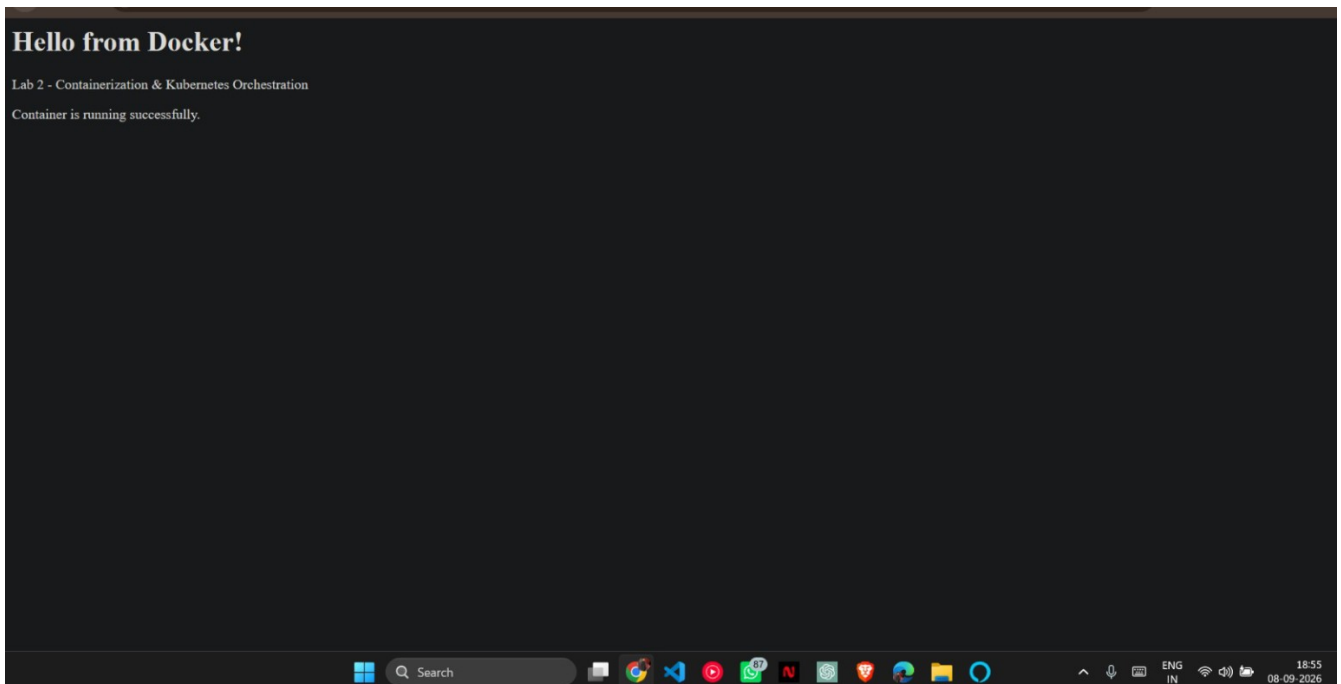


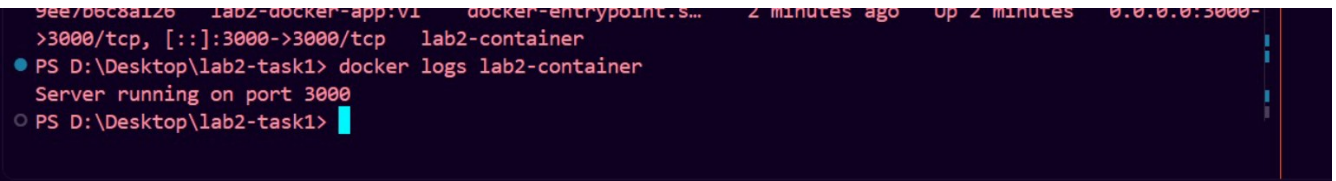
Figure 2. Successful Docker image build using `docker build -t lab2-docker-app:v1 ..`

Result

The build output shows all build steps finishing successfully and the resulting image being named **lab2-docker-app:v1**.

3. Docker Image and Layer Documentation

The image was verified with **docker images**. The image history was then inspected with **docker history lab2-docker-app:v1** to document the image layers generated by the Dockerfile and inherited from the base image.



```
3e16a9034e4a lab2-docker-app:v1 docker-entrypoint.s... 2 minutes ago Up 2 minutes 0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp lab2-container
PS D:\Desktop\lab2-task1> docker logs lab2-container
Server running on port 3000
PS D:\Desktop\lab2-task1>
```

Figure 3. Docker image listing and image history showing commands/layers such as WORKDIR, COPY, EXPOSE and CMD, together with inherited Node/Alpine layers.

Observed image information

The image listing shows **lab2-docker-app:v1** with image ID beginning **3e16a9034e4a**. The history output documents the application-specific layers, including **WORKDIR /app**, **COPY package.json .**, **COPY app.js .**, **EXPOSE 3000/tcp**, and **CMD ["node", "app.js"]**, along with the layers inherited from the Node Alpine base image.

4. Running the Docker Container

The built image was run as a detached container named **lab2-container**. Port 3000 on the host was mapped to port 3000 in the container.

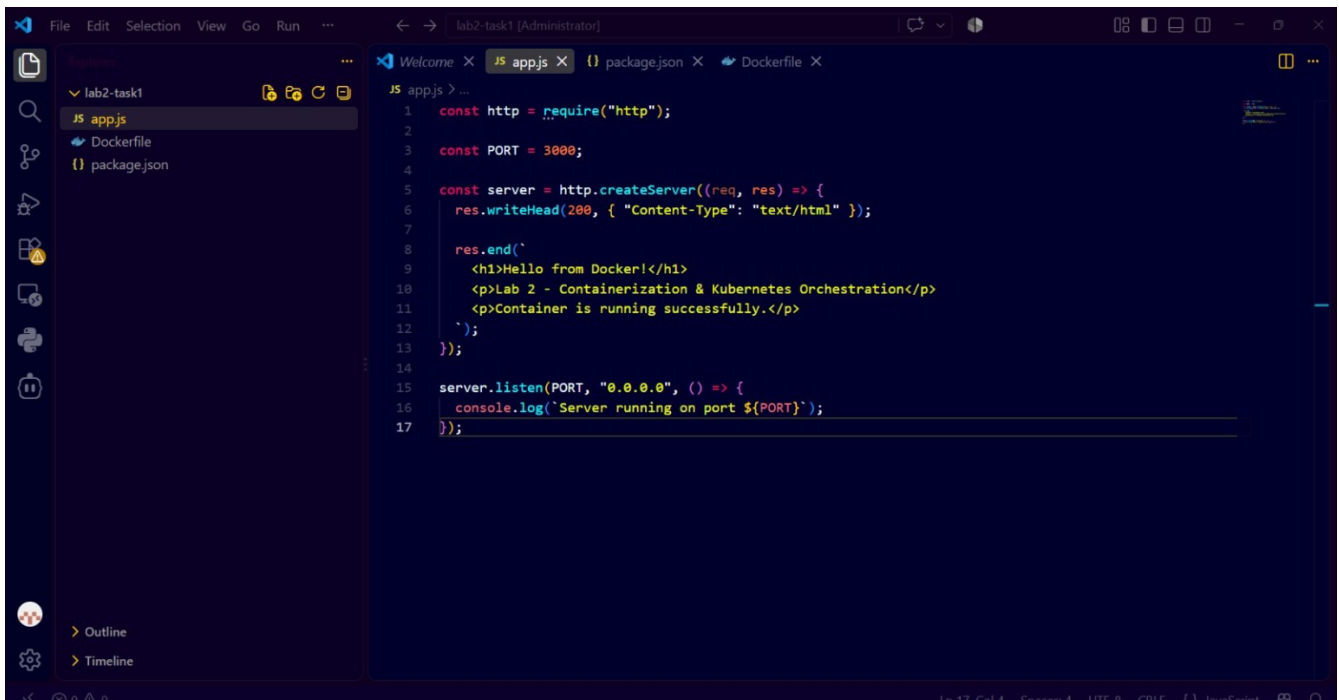


Figure 4. Container execution and verification using **docker ps** / container port mapping.

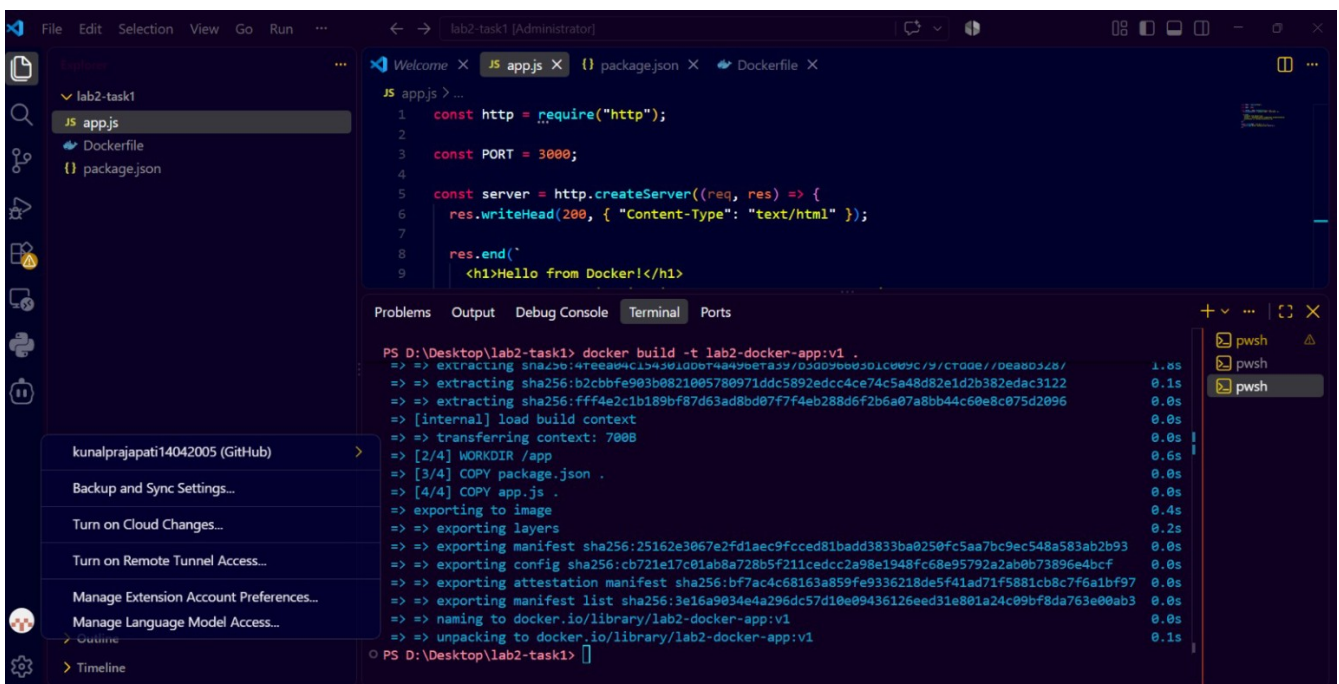


Figure 5. Container execution and verification using **docker ps** / container port mapping.

Container behavior

The **docker ps** output confirms that the container is running. The port mapping is shown as **0.0.0.0:3000->3000/tcp** (and the IPv6 equivalent), making the application available on the host through port 3000.

5. Application Verification in the Browser

After the container was started, the published port was accessed in a browser. The application responded successfully with the expected HTML content.

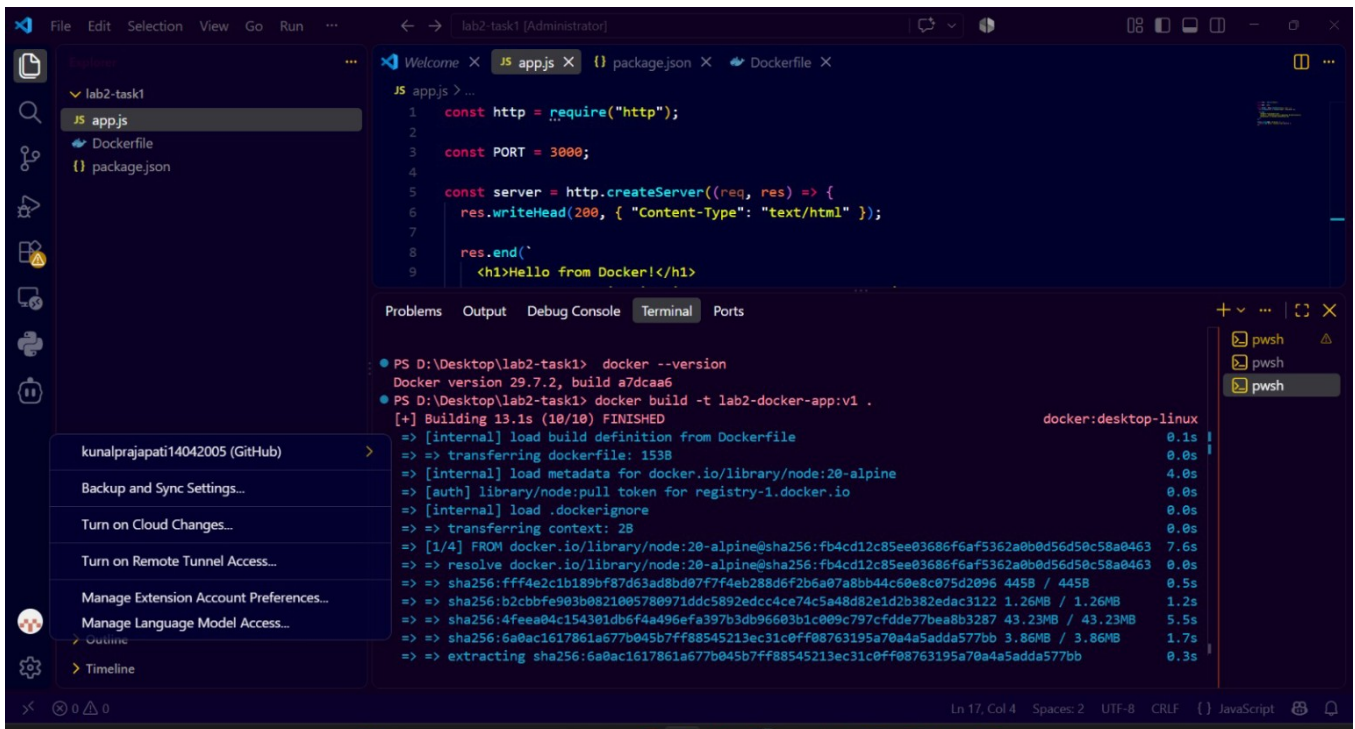


Figure 6. Browser verification showing "Hello from Docker!" and "Container is running successfully."

Verification result

The browser output confirms that the Node.js application is reachable through the Docker-published port and is serving the expected response from inside the container.

6. Container Logs and Runtime Evidence

The container logs were checked using **docker logs lab2-container**. The application printed its startup message, confirming that the Node.js server initialized successfully.

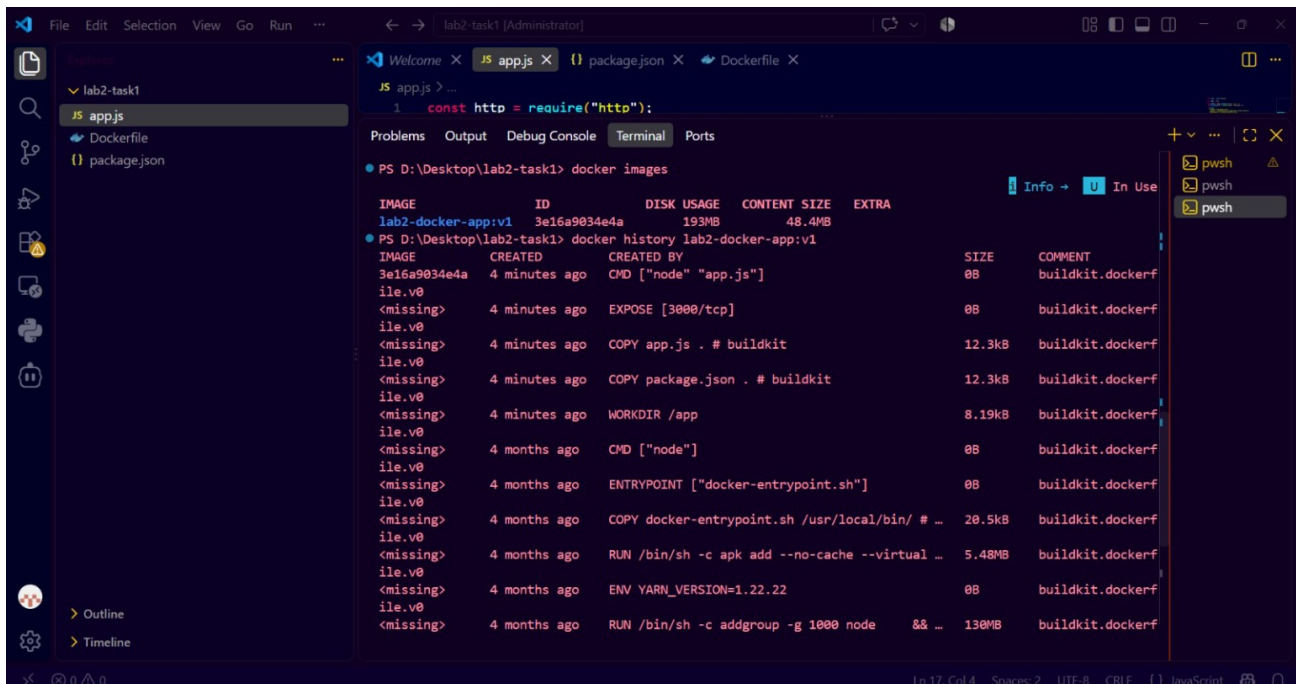


Figure 7. Docker runtime evidence: container status and server log output.

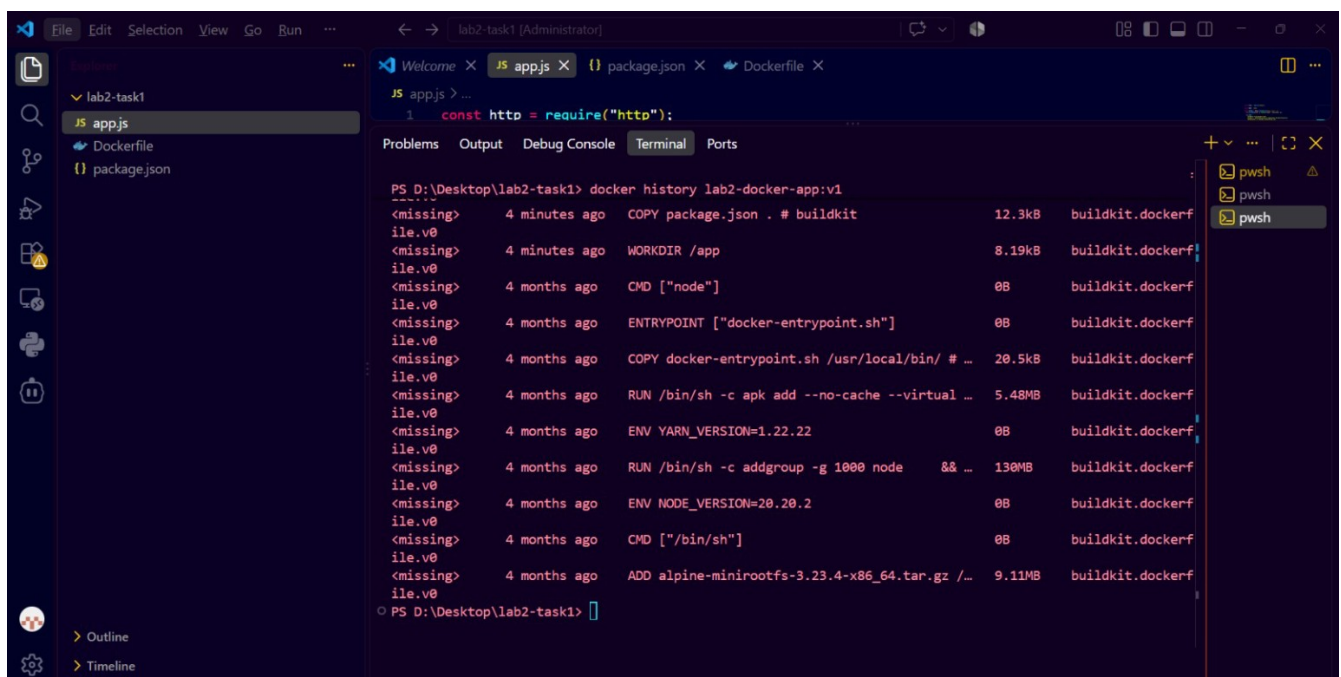


Figure 8. Docker runtime evidence: container status and server log output.

Observed log

Server running on port 3000

7. Task 1 Requirement Checklist

Requirement	Evidence / Result	Status
Write a Dockerfile	Dockerfile present in the project and used for the build.	Completed
Build the image	docker build completed successfully; image tagged lab2-docker-app:v1	Completed
Run the container	lab2-container was started successfully.	Completed
Document image layers	docker history lab2-docker-app:v1 captured application and base-image layers	Completed
Document container behavior	docker ps, browser output and docker logs verify runtime behavior.	Completed

Conclusion

Task 1 was completed by containerizing the sample Node.js application with Docker, building and inspecting the image, running the container with port mapping, and verifying the application through both browser output and container logs. The screenshots in this report provide the corresponding execution evidence.

Note for submission: Upload this PDF as the single file for Task 1. Keep the project source files (app.js, Dockerfile, package.json, and any Compose file used later) in the GitHub repository separately.